

IMPLEMENTING AND OPTIMIZING THE SHORTEST VECTOR PROBLEM

FREDDY CUNNINGHAM

1. APPROACH

I chose to use the Schnorr–Euchner Enumeration algorithm [1] for my approach to solve the SVP problem as it provides an exact solution and is reasonably simple to implement. Enumeration is a polynomial-space algorithm using less memory than the other main approach sieving which is an exponential-space algorithm. However, enumeration has super-exponential run-time [2] whereas sieving has single exponential run-time [3]. For this coursework our implementation will only be evaluated on inputs up to 10 dimensions so enumeration is a natural choice as in practise it typically has good performance on low dimensions.

The run-time of enumeration algorithms is greatly affected by the quality of the input basis. A better basis is shorter and more orthogonal [4]. I therefore chose to implement the LLL basis reduction algorithm to preprocess the input basis before running the enumeration algorithm. I use the norm of the first vector in the reduced basis as the bound for the search space used by the enumeration algorithm.

2. IMPLEMENTATION

I chose to implement my final solution in C. The primary components of the program are the Gram Schmidt, LLL and Enumeration functions. The Gram Schmidt algorithm is implemented using floating-point arithmetic but initially I considered implementing the algorithm using exact arithmetic, storing the values as fractions to remove the floating point errors. However, I decided against this decision as the lost precision due to using floating-point arithmetic was negligible in comparison to the increase in run-time.

All numerical values were stored as doubles as we are required to be able to handle at least 32-bit floating point precision as per the coursework. All vectors were stored as C arrays and matrices as 2D C arrays.

The classical Gram-Schmidt algorithm I implemented is numerically unstable which means errors are introduced to the orthogonalised vectors and Gram-Schmidt coefficients due to the finite-precision of float-point arithmetic. These errors are particularly prevalent when the vectors are almost colinear

3. RUN-TIME

To test my implementation I generated uniform and knapsack like matrices of dimension 1×1 to 40×40 containing numbers of size 8, 16 and 32 bits.

To record the run-time I used hyperfine a command-line benchmarking tool.

The tests were performed on a virtual machine with 4 cores and 32 GB RAM running an Ubuntu Linux image.

4. MEMORY

5. ACCURACY

To determine the accuracy of my implementation I compared my solution to the solution generated by the `fpvll` library. Given that I am working with floating-point arithmetic it is likely that I encounter small differences between the my solution and the reference solution. Therefore, I decided to allow a small tolerance level of 0.00001 that my solution must be within to pass.

REFERENCES

1. Yasuda, M.: A survey of solving SVP algorithms and recent strategies for solving the SVP Challenge. International Symposium on Mathematics, Quantum Theory, and Cryptography. 189 (2020). https://doi.org/10.1007/978-981-15-5191-8_15
2. Gama, P. Q. A. R. O., Nicolas AND Nguyen: Lattice enumeration using extreme pruning. Advances in Cryptology – EUROCRYPT 2010. 257–278 (2010). https://doi.org/10.1007/978-3-642-13190-5_13
3. Ajtai, R. A. S. D., Miklós AND Kumar: A sieve algorithm for the shortest lattice vector problem. Proceedings of the thirty-third annual ACM symposium on Theory of computing. (2001). <https://doi.org/10.1145/380752.380857>
4. Kumar, M.: Faster Lattice Enumeration. CoRR. (2019)

DEPARTMENT OF COMPUTER SCIENCE, DURHAM UNIVERSITY
Email address: freddy.t.cunningham@durham.ac.uk